

Section 92 Editorial — Discrete Logarithms And Modular Roots

A. Smallest Primitive Root

How to find it

Factor $p-1$ and reject a candidate if removing any distinct prime factor from the exponent makes it one.

Correctness

The order divides $p-1$; it is proper exactly when it divides $(p-1)/q$ for some prime divisor q .

Complexity

Factorization plus $O(\text{number tested} * \omega(p-1) \log p)$.

C++

```
#include <bits/stdc++.h>
using namespace std;
using u64=uint64_t;
using u128=__uint128_t;
using i128=__int128_t;
u64 mul(u64 a,u64 b,u64 m) {
    return(u128)a*b%m;
}
u64 power(u64 a,u64 e,u64 m) {
    u64 r=1%m;
    for(;e>=1,a=mul(a,a,m))if(e&1)r=mul(r,a,m);
    return r;
}
bool prime(u64 n) {
    if(n<2)return false;
    for(u64 p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
    u64 d=n-1,s=0;
    while(!(d&1))d>>=1,++s;
    for(u64 a:{2,325,9375,28178,450775,9780504,1795265022}) {
        if(a%n==0)continue;
        u64 x=power(a,d,n);
        if(x==1||x==n-1)continue;
        bool witness=true;
        for(u64 r=1;r<s;r++) {
            x=mul(x,x,n);
            if(x==n-1) {
                witness=false;
                break;
            }
        }
        if(witness)return false;
    }
    return true;
}
```

```

mt19937_64 rng(712367821);
u64 rho(u64 n) {
    if(n%2==0)return 2;
    if(n%3==0)return 3;
    while(true) {
        u64 c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
        while(d==1) {
            x=(mul(x,x,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            d=gcd(x>y?x-y:y-x,n);
        }
        if(d!=n)return d;
    }
}
vector<u64> factor(u64 n) {
    vector<u64>out,st= {
        n
    };
    ;
    while(!st.empty()) {
        u64 x=st.back();
        st.pop_back();
        if(x==1)continue;
        if(prime(x))out.push_back(x);
        else {
            u64 d=rho(x);
            st.push_back(d);
            st.push_back(x/d);
        }
    }
    sort(out.begin(),out.end());
    return out;
}
vector<pair<u64,int>>groups(u64 n) {
    vector<pair<u64,int>>g;
    for(u64 p:factor(n))if(!g.empty()&&g.back().first==p)g.back().second++;
    else g.push_back({p,1});
    return g;
}
u64 phi(u64 n) {
    u64 r=n;
    for(auto[p,e]:groups(n))r=r/p*(p-1);
    return r;
}
u64 lcm64(u64 a,u64 b) {
    return a/gcd(a,b)*b;
}
u64 lambda(u64 n) {
    u64 r=1;
    for(auto[p,e]:groups(n)) {
        u64 z;
        if(p==2&&e>=3)z=1ULL<<(e-2);
        else {
            z=p-1;
            for(int i=1;i<e;i++)z*=p;
        }
        r=lcm64(r,z);
    }
    return r;
}
u64 primitive(u64 p) {
    if(p==2)return 1;

```

```

vector<u64>q;
for(auto [x,e]:groups(p-1))q.push_back(x);
for(u64 g=2;;g++) {
    bool ok=true;
    for(u64 x:q)if(power(g,(p-1)/x,p)==1) {
        ok=false;
        break;
    }
    if(ok)return g;
}
}

long long invmod(long long a,long long m) {
    long long b=m;
    i128 x=1,y=0;
    while(b) {
        long long q=a/b,t=a%b;
        a=b;
        b=t;
        i128 z=x-(i128)q*y;
        x=y;
        y=z;
    }
    x%=m;
    if(x<0)x+=m;
    return(long long)x;
}

optional<u64>bsgs(u64 a,u64 b,u64 m) {
    if(m==1)return 0;
    u64 z=sqrtl((long double)m)+1;
    while((u128)z*z<m)z++;
    unordered_map<u64,u64>baby;
    u64 value=1;
    for(u64 j=0;j<z;j++) {
        if(!baby.count(value))baby[value]=j;
        value=mul(value,a,m);
    }
    u64 step=invmod(power(a,z,m),m);
    value=b;
    optional<u64>best;
    for(u64 i=0;i<=z;i++) {
        auto it=baby.find(value);
        if(it!=baby.end()) {
            u64 candidate=i*z+it->second;
            if(!best||candidate<*best)best=candidate;
        }
        value=mul(value,step,m);
    }
    return best;
}

optional<u64>dlog(u64 a,u64 b,u64 m) {
    a%=m;
    b%=m;
    if(b==1%m)return 0;
    u64 added=0,scale=1;
    while(true) {
        u64 g=gcd(a,m);
        if(g==1)break;
        if(b%g)return {
        }
        ;
        m/=g;
        b/=g;
        scale=mul(scale,a/g,m);
    }
}

```

```

        added++;
        if(scale==b)return added;
        if(m==1)return added;
    }
    u64 target=mul(b,invmod(scale,m),m);
    auto tail=bsgs(a%m,target,m);
    if(!tail)return {
    }
    ;
    return added+*tail;
}

optional<u64>tonelli(u64 a,u64 p) {
    a%=p;
    if(a==0)return 0;
    if(power(a,(p-1)/2,p)!=1)return {
    }
    ;
    if(p%4==3)return power(a,(p+1)/4,p);
    u64 q=p-1,s=0;
    while(!(q&1))q>>=1,++s;
    u64 z=2;
    while(power(z,(p-1)/2,p)!=p-1)z++;
    u64 c=power(z,q,p),x=power(a,(q+1)/2,p),t=power(a,q,p),m=s;
    while(t!=1) {
        u64 i=1,v=mul(t,t,p);
        while(v!=1)v=mul(v,v,p),i++;
        u64 h=power(c,1ULL<<(m-i-1),p);
        x=mul(x,h,p);
        t=mul(t,mul(h,h,p),p);
        c=mul(h,h,p);
        m=i;
    }
    return x;
}

u64 order(u64 a,u64 n) {
    u64 r=lambda(n);
    for(auto [p,e]:groups(r))while(r%p==0&&power(a,r/p,n)==1)r/=p;
    return r;
}

optional<u64>linear(u64 a,u64 b,u64 m) {
    u64 g=gcd(a,m);
    if(b%g)return {
    }
    ;
    u64 mod=m/g;
    if(mod==1)return 0;
    return mul((b/g)%mod,invmod((a/g)%mod,mod),mod);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        u64 p;
        cin>>p;
        cout<<primitive(p)<<'\\n';
    }
}

```

Python

```
import sys, math, random
MR_BASES = (2, 325, 9375, 28178, 450775, 9780504, 1795265022)

def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2
    for a in MR_BASES:
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
_rng = random.Random(712367821)

def rho(n):
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3
    while True:
        c = _rng.randrange(1, n)
        x = _rng.randrange(0, n)
        y = x
        d = 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = math.gcd(abs(x - y), n)
        if d != n:
            return d

def factor(n):
    out = []
    stack = [n]
    while stack:
        x = stack.pop()
        if x == 1:
            continue
        if is_prime(x):
            out.append(x)
        else:
            d = rho(x)
            stack.extend((d, x // d))
    out.sort()
    return out
```

```

def groups(n):
    result = []
    for p in factor(n):
        if result and result[-1][0] == p:
            result[-1] = (p, result[-1][1] + 1)
        else:
            result.append((p, 1))
    return result

def phi(n):
    answer = n
    for p, _ in groups(n):
        answer = answer // p * (p - 1)
    return answer

def lcm(a, b):
    return a // math.gcd(a, b) * b

def carmichael(n):
    answer = 1
    for p, e in groups(n):
        part = 1 << e - 2 if p == 2 and e >= 3 else (p - 1) * p ** (e - 1)
        answer = lcm(answer, part)
    return answer

def primitive_root(p):
    if p == 2:
        return 1
    primes = [q for q, _ in groups(p - 1)]
    g = 2
    while any((pow(g, (p - 1) // q, p) == 1 for q in primes)):
        g += 1
    return g

def bsgs(a, b, m):
    a %= m
    b %= m
    if m == 1:
        return 0
    size = math.isqrt(m) + 1
    baby = {}
    value = 1
    for j in range(size):
        baby.setdefault(value, j)
        value = value * a % m
    step = pow(pow(a, size, m), -1, m)
    value = b
    best = None
    for i in range(size + 1):
        if value in baby:
            candidate = i * size + baby[value]
            if best is None or candidate < best:
                best = candidate
        value = value * step % m
    return best

def discrete_log(a, b, m):
    a %= m
    b %= m
    if b == 1 % m:
        return 0
    added = 0
    scale = 1

```

```

while True:
    g = math.gcd(a, m)
    if g == 1:
        break
    if b % g:
        return None
    m //= g
    b //= g
    scale = scale * (a // g) % m
    added += 1
    if scale == b:
        return added
    if m == 1:
        return added
target = b * pow(scale, -1, m) % m
tail = bsgs(a % m, target, m)
return None if tail is None else added + tail

def tonelli(a, p):
    a %= p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    q = p - 1
    s = 0
    while q % 2 == 0:
        s += 1
        q //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    c = pow(z, q, p)
    x = pow(a, (q + 1) // 2, p)
    t = pow(a, q, p)
    m = s
    while t != 1:
        i = 1
        value = t * t % p
        while value != 1:
            value = value * value % p
            i += 1
        b = pow(c, 1 << m - i - 1, p)
        x = x * b % p
        t = t * b * b % p
        c = b * b % p
        m = i
    return x

def order(a, n):
    value = carmichael(n)
    for p, _ in groups(value):
        while value % p == 0 and pow(a, value // p, n) == 1:
            value //= p
    return value

def linear_smallest(a, b, m):
    g = math.gcd(a, m)
    if b % g:
        return None
    mod = m // g

```

```

    return b // g * pow(a // g, -1, mod) % mod if mod > 1 else 0

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    print('\n'.join((str(primitive_root(p)) for p in d[1:])))
if __name__ == '__main__':
    main()

```

B. Prime Discrete Log

How to find it

Meet baby exponents and giant exponent blocks.

Correctness

A table match is algebraically equivalent to the target power equation, and every exponent has one such split.

Complexity

$O(\sqrt{p})$ time and memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
using u64=uint64_t;
using u128=__uint128_t;
using i128=__int128_t;
u64 mul(u64 a,u64 b,u64 m) {
    return(u128)a*b%m;
}
u64 power(u64 a,u64 e,u64 m) {
    u64 r=1%m;
    for(;e>=>1,a=mul(a,a,m))if(e&1)r=mul(r,a,m);
    return r;
}
bool prime(u64 n) {
    if(n<2)return false;
    for(u64 p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
    u64 d=n-1,s=0;
    while(!(d&1))d>=>1,++s;
    for(u64 a:{2,325,9375,28178,450775,9780504,1795265022}) {
        if(a%n==0)continue;
        u64 x=power(a,d,n);
        if(x==1||x==n-1)continue;
        bool witness=true;
        for(u64 r=1;r<s;r++) {
            x=mul(x,x,n);
            if(x==n-1) {
                witness=false;
                break;
            }
        }
        if(witness)return false;
    }
    return true;
}
mt19937_64 rng(712367821);

```

```

u64 rho(u64 n) {
    if(n%2==0)return 2;
    if(n%3==0)return 3;
    while(true) {
        u64 c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
        while(d==1) {
            x=(mul(x,x,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            d=gcd(x>y?x-y:y-x,n);
        }
        if(d!=n)return d;
    }
}

vector<u64> factor(u64 n) {
    vector<u64>out,st= {
        n
    }
    ;
    while(!st.empty()) {
        u64 x=st.back();
        st.pop_back();
        if(x==1)continue;
        if(primex())out.push_back(x);
        else {
            u64 d=rho(x);
            st.push_back(d);
            st.push_back(x/d);
        }
    }
    sort(out.begin(),out.end());
    return out;
}

vector<pair<u64,int>>groups(u64 n) {
    vector<pair<u64,int>>g;
    for(u64 p:factor(n))if(!g.empty()&&g.back().first==p)g.back().second++;
    else g.push_back({p,1});
    return g;
}

u64 phi(u64 n) {
    u64 r=n;
    for(auto[p,e]:groups(n))r=r/p*(p-1);
    return r;
}

u64 lcm64(u64 a,u64 b) {
    return a/gcd(a,b)*b;
}

u64 lambda(u64 n) {
    u64 r=1;
    for(auto[p,e]:groups(n)) {
        u64 z;
        if(p==2&&e>=3)z=1ULL<<(e-2);
        else {
            z=p-1;
            for(int i=1;i<e;i++)z*=p;
        }
        r=lcm64(r,z);
    }
    return r;
}

u64 primitive(u64 p) {
    if(p==2)return 1;
    vector<u64>q;

```

```

for(auto[x,e]:groups(p-1))q.push_back(x);
for(u64 g=2;;g++) {
    bool ok=true;
    for(u64 x:q)if(power(g,(p-1)/x,p)==1) {
        ok=false;
        break;
    }
    if(ok)return g;
}
}

long long invmod(long long a,long long m) {
    long long b=m;
    i128 x=1,y=0;
    while(b) {
        long long q=a/b,t=a%b;
        a=b;
        b=t;
        i128 z=x-(i128)q*y;
        x=y;
        y=z;
    }
    x%=m;
    if(x<0)x+=m;
    return(long long)x;
}

optional<u64>bsgs(u64 a,u64 b,u64 m) {
    if(m==1)return 0;
    u64 z=sqrtl((long double)m)+1;
    while((u128)z*z<m)z++;
    unordered_map<u64,u64>baby;
    u64 value=1;
    for(u64 j=0;j<z;j++) {
        if(!baby.count(value))baby[value]=j;
        value=mul(value,a,m);
    }
    u64 step=invmod(power(a,z,m),m);
    value=b;
    optional<u64>best;
    for(u64 i=0;i<=z;i++) {
        auto it=baby.find(value);
        if(it!=baby.end()) {
            u64 candidate=i*z+it->second;
            if(!best||candidate<*best)best=candidate;
        }
        value=mul(value,step,m);
    }
    return best;
}

optional<u64>dlog(u64 a,u64 b,u64 m) {
    a%=m;
    b%=m;
    if(b==1%m)return 0;
    u64 added=0,scale=1;
    while(true) {
        u64 g=gcd(a,m);
        if(g==1)break;
        if(b%g)return {
        }
        ;
        m/=g;
        b/=g;
        scale=mul(scale,a/g,m);
        added++;
    }
}

```

```

        if(scale==b)return added;
        if(m==1)return added;
    }
    u64 target=mul(b,invmod(scale,m),m);
    auto tail=bsgs(a%m,target,m);
    if(!tail)return {
    }
    ;
    return added+*tail;
}
optional<u64>tonelli(u64 a,u64 p) {
    a%=p;
    if(a==0)return 0;
    if(power(a,(p-1)/2,p)!=1)return {
    }
    ;
    if(p%4==3)return power(a,(p+1)/4,p);
    u64 q=p-1,s=0;
    while(!(q&1))q>>=1,++s;
    u64 z=2;
    while(power(z,(p-1)/2,p)!=p-1)z++;
    u64 c=power(z,q,p),x=power(a,(q+1)/2,p),t=power(a,q,p),m=s;
    while(t!=1) {
        u64 i=1,v=mul(t,t,p);
        while(v!=1)v=mul(v,v,p),i++;
        u64 h=power(c,1ULL<<(m-i-1),p);
        x=mul(x,h,p);
        t=mul(t,mul(h,h,p),p);
        c=mul(h,h,p);
        m=i;
    }
    return x;
}
u64 order(u64 a,u64 n) {
    u64 r=lambda(n);
    for(auto [p,e]:groups(r))while(r%p==0&&power(a,r/p,n)==1)r/=p;
    return r;
}
optional<u64>linear(u64 a,u64 b,u64 m) {
    u64 g=gcd(a,m);
    if(b%g)return {
    }
    ;
    u64 mod=m/g;
    if(mod==1)return 0;
    return mul((b/g)%mod,invmod((a/g)%mod,mod),mod);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        u64 p,a,b;
        cin>>p>>a>>b;
        auto x=dlog(a,b,p);
        if(x)cout<<*x<<'\\n';
        else cout<<"-1\\n";
    }
}

```

Python

```
import sys, math, random
MR_BASES = (2, 325, 9375, 28178, 450775, 9780504, 1795265022)

def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2
    for a in MR_BASES:
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
_rng = random.Random(712367821)

def rho(n):
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3
    while True:
        c = _rng.randrange(1, n)
        x = _rng.randrange(0, n)
        y = x
        d = 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = math.gcd(abs(x - y), n)
        if d != n:
            return d

def factor(n):
    out = []
    stack = [n]
    while stack:
        x = stack.pop()
        if x == 1:
            continue
        if is_prime(x):
            out.append(x)
        else:
            d = rho(x)
            stack.extend((d, x // d))
    out.sort()
    return out
```

```

def groups(n):
    result = []
    for p in factor(n):
        if result and result[-1][0] == p:
            result[-1] = (p, result[-1][1] + 1)
        else:
            result.append((p, 1))
    return result

def phi(n):
    answer = n
    for p, _ in groups(n):
        answer = answer // p * (p - 1)
    return answer

def lcm(a, b):
    return a // math.gcd(a, b) * b

def carmichael(n):
    answer = 1
    for p, e in groups(n):
        part = 1 << e - 2 if p == 2 and e >= 3 else (p - 1) * p ** (e - 1)
        answer = lcm(answer, part)
    return answer

def primitive_root(p):
    if p == 2:
        return 1
    primes = [q for q, _ in groups(p - 1)]
    g = 2
    while any((pow(g, (p - 1) // q, p) == 1 for q in primes)):
        g += 1
    return g

def bsgs(a, b, m):
    a %= m
    b %= m
    if m == 1:
        return 0
    size = math.isqrt(m) + 1
    baby = {}
    value = 1
    for j in range(size):
        baby.setdefault(value, j)
        value = value * a % m
    step = pow(pow(a, size, m), -1, m)
    value = b
    best = None
    for i in range(size + 1):
        if value in baby:
            candidate = i * size + baby[value]
            if best is None or candidate < best:
                best = candidate
        value = value * step % m
    return best

def discrete_log(a, b, m):
    a %= m
    b %= m
    if b == 1 % m:
        return 0
    added = 0
    scale = 1

```

```

while True:
    g = math.gcd(a, m)
    if g == 1:
        break
    if b % g:
        return None
    m //= g
    b //= g
    scale = scale * (a // g) % m
    added += 1
    if scale == b:
        return added
    if m == 1:
        return added
target = b * pow(scale, -1, m) % m
tail = bsgs(a % m, target, m)
return None if tail is None else added + tail

def tonelli(a, p):
    a %= p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    q = p - 1
    s = 0
    while q % 2 == 0:
        s += 1
        q //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    c = pow(z, q, p)
    x = pow(a, (q + 1) // 2, p)
    t = pow(a, q, p)
    m = s
    while t != 1:
        i = 1
        value = t * t % p
        while value != 1:
            value = value * value % p
            i += 1
        b = pow(c, 1 << m - i - 1, p)
        x = x * b % p
        t = t * b * b % p
        c = b * b % p
        m = i
    return x

def order(a, n):
    value = carmichael(n)
    for p, _ in groups(value):
        while value % p == 0 and pow(a, value // p, n) == 1:
            value //= p
    return value

def linear_smallest(a, b, m):
    g = math.gcd(a, m)
    if b % g:
        return None
    mod = m // g

```

```

    return b // g * pow(a // g, -1, mod) % mod if mod > 1 else 0

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(1, len(d), 3):
        p, a, b = d[i:i + 3]
        x = discrete_log(a, b, p)
        out.append(str(-1 if x is None else x))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

C. General Discrete Log

How to find it

Strip common gcd factors until modular inverses become legal, then run BSGS on the adjusted equation.

Correctness

Each stripping step is necessary and equivalent after accounting for its forced base factor; the final coprime equation is handled exhaustively by BSGS.

Complexity

$O(\log m + \sqrt{m})$ expected time and $O(\sqrt{m})$ memory.

C++

```

#include <bits/stdc++.h>
using namespace std;
using u64=uint64_t;
using u128=__uint128_t;
using i128=__int128_t;
u64 mul(u64 a,u64 b,u64 m) {
    return(u128)a*b%m;
}
u64 power(u64 a,u64 e,u64 m) {
    u64 r=1%m;
    for(;e>=1,a=mul(a,a,m))if(e&1)r=mul(r,a,m);
    return r;
}
bool prime(u64 n) {
    if(n<2)return false;
    for(u64 p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
    u64 d=n-1,s=0;
    while(!(d&1))d>=1,++s;
    for(u64 a:{2,325,9375,28178,450775,9780504,1795265022}) {
        if(a%n==0)continue;
        u64 x=power(a,d,n);
        if(x==1||x==n-1)continue;
        bool witness=true;
        for(u64 r=1;r<s;r++) {
            x=mul(x,x,n);
            if(x==n-1) {
                witness=false;
                break;
            }
        }
    }
}

```

```

    }
    if(witness)return false;
}
return true;
}
mt19937_64 rng(712367821);
u64 rho(u64 n) {
    if(n%2==0)return 2;
    if(n%3==0)return 3;
    while(true) {
        u64 c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
        while(d==1) {
            x=(mul(x,x,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            d=gcd(x>y?x-y:y-x,n);
        }
        if(d!=n)return d;
    }
}
vector<u64> factor(u64 n) {
    vector<u64>out,st= {
        n
    }
    ;
    while(!st.empty()) {
        u64 x=st.back();
        st.pop_back();
        if(x==1)continue;
        if(prime(x))out.push_back(x);
        else {
            u64 d=rho(x);
            st.push_back(d);
            st.push_back(x/d);
        }
    }
    sort(out.begin(),out.end());
    return out;
}
vector<pair<u64,int>>groups(u64 n) {
    vector<pair<u64,int>>g;
    for(u64 p:factor(n))if(!g.empty()&&g.back().first==p)g.back().second++;
    else g.push_back({p,1});
    return g;
}
u64 phi(u64 n) {
    u64 r=n;
    for(auto[p,e]:groups(n))r=r/p*(p-1);
    return r;
}
u64 lcm64(u64 a,u64 b) {
    return a/gcd(a,b)*b;
}
u64 lambda(u64 n) {
    u64 r=1;
    for(auto[p,e]:groups(n)) {
        u64 z;
        if(p==2&&e>=3)z=1ULL<<(e-2);
        else {
            z=p-1;
            for(int i=1;i<e;i++)z*=p;
        }
        r=lcm64(r,z);
    }
}

```

```

    }
    return r;
}
u64 primitive(u64 p) {
    if(p==2)return 1;
    vector<u64>q;
    for(auto [x,e]:groups(p-1))q.push_back(x);
    for(u64 g=2;;g++) {
        bool ok=true;
        for(u64 x:q)if(power(g,(p-1)/x,p)==1) {
            ok=false;
            break;
        }
        if(ok)return g;
    }
}
long long invmod(long long a,long long m) {
    long long b=m;
    i128 x=1,y=0;
    while(b) {
        long long q=a/b,t=a%b;
        a=b;
        b=t;
        i128 z=x-(i128)q*y;
        x=y;
        y=z;
    }
    x%=m;
    if(x<0)x+=m;
    return(long long)x;
}
optional<u64>bsgs(u64 a,u64 b,u64 m) {
    if(m==1)return 0;
    u64 z=sqrtl((long double)m)+1;
    while((u128)z*z<m)z++;
    unordered_map<u64,u64>baby;
    u64 value=1;
    for(u64 j=0;j<z;j++) {
        if(!baby.count(value))baby[value]=j;
        value=mul(value,a,m);
    }
    u64 step=invmod(power(a,z,m),m);
    value=b;
    optional<u64>best;
    for(u64 i=0;i<=z;i++) {
        auto it=baby.find(value);
        if(it!=baby.end()) {
            u64 candidate=i*z+it->second;
            if(!best||candidate<*best)best=candidate;
        }
        value=mul(value,step,m);
    }
    return best;
}
optional<u64>dlog(u64 a,u64 b,u64 m) {
    a%=m;
    b%=m;
    if(b==1%m)return 0;
    u64 added=0,scale=1;
    while(true) {
        u64 g=gcd(a,m);
        if(g==1)break;
        if(b%g)return {

```

```

    }
    ;
    m/=g;
    b/=g;
    scale=mul(scale,a/g,m);
    added++;
    if(scale==b)return added;
    if(m==1)return added;
}
u64 target=mul(b,invmod(scale,m),m);
auto tail=bsgs(a%m,target,m);
if(!tail)return {
}
;
return added+*tail;
}
optional<u64>tonelli(u64 a,u64 p) {
    a%=p;
    if(a==0)return 0;
    if(power(a,(p-1)/2,p)!=1)return {
    }
    ;
    if(p%4==3)return power(a,(p+1)/4,p);
    u64 q=p-1,s=0;
    while(!(q&1))q>>=1,++s;
    u64 z=2;
    while(power(z,(p-1)/2,p)!=p-1)z++;
    u64 c=power(z,q,p),x=power(a,(q+1)/2,p),t=power(a,q,p),m=s;
    while(t!=1) {
        u64 i=1,v=mul(t,t,p);
        while(v!=1)v=mul(v,v,p),i++;
        u64 h=power(c,1ULL<<(m-i-1),p);
        x=mul(x,h,p);
        t=mul(t,mul(h,h,p),p);
        c=mul(h,h,p);
        m=i;
    }
    return x;
}
u64 order(u64 a,u64 n) {
    u64 r=lambda(n);
    for(auto [p,e]:groups(r))while(r%p==0&&power(a,r/p,n)==1)r/=p;
    return r;
}
optional<u64>linear(u64 a,u64 b,u64 m) {
    u64 g=gcd(a,m);
    if(b%g)return {
    }
    ;
    u64 mod=m/g;
    if(mod==1)return 0;
    return mul((b/g)%mod,invmod((a/g)%mod,mod),mod);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;
    while(q--) {
        u64 a,b,m;
        cin>>a>>b>>m;
        auto x=dlog(a,b,m);
        if(x)cout<<*x<<'\\n';
    }
}

```

```

        else cout<<"-1\n";
    }
}

```

Python

```

import sys, math, random
MR_BASES = (2, 325, 9375, 28178, 450775, 9780504, 1795265022)

def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2
    for a in MR_BASES:
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
_rng = random.Random(712367821)

def rho(n):
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3
    while True:
        c = _rng.randrange(1, n)
        x = _rng.randrange(0, n)
        y = x
        d = 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = math.gcd(abs(x - y), n)
        if d != n:
            return d

def factor(n):
    out = []
    stack = [n]
    while stack:
        x = stack.pop()
        if x == 1:
            continue
        if is_prime(x):
            out.append(x)

```

```

        else:
            d = rho(x)
            stack.extend((d, x // d))
    out.sort()
    return out

def groups(n):
    result = []
    for p in factor(n):
        if result and result[-1][0] == p:
            result[-1] = (p, result[-1][1] + 1)
        else:
            result.append((p, 1))
    return result

def phi(n):
    answer = n
    for p, _ in groups(n):
        answer = answer // p * (p - 1)
    return answer

def lcm(a, b):
    return a // math.gcd(a, b) * b

def carmichael(n):
    answer = 1
    for p, e in groups(n):
        part = 1 << e - 2 if p == 2 and e >= 3 else (p - 1) * p ** (e - 1)
        answer = lcm(answer, part)
    return answer

def primitive_root(p):
    if p == 2:
        return 1
    primes = [q for q, _ in groups(p - 1)]
    g = 2
    while any((pow(g, (p - 1) // q, p) == 1 for q in primes)):
        g += 1
    return g

def bsgs(a, b, m):
    a %= m
    b %= m
    if m == 1:
        return 0
    size = math.isqrt(m) + 1
    baby = {}
    value = 1
    for j in range(size):
        baby.setdefault(value, j)
        value = value * a % m
    step = pow(pow(a, size, m), -1, m)
    value = b
    best = None
    for i in range(size + 1):
        if value in baby:
            candidate = i * size + baby[value]
            if best is None or candidate < best:
                best = candidate
        value = value * step % m
    return best

def discrete_log(a, b, m):

```

```

a %= m
b %= m
if b == 1 % m:
    return 0
added = 0
scale = 1
while True:
    g = math.gcd(a, m)
    if g == 1:
        break
    if b % g:
        return None
    m //= g
    b //= g
    scale = scale * (a // g) % m
    added += 1
    if scale == b:
        return added
    if m == 1:
        return added
target = b * pow(scale, -1, m) % m
tail = bsgs(a % m, target, m)
return None if tail is None else added + tail

def tonelli(a, p):
    a %= p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    q = p - 1
    s = 0
    while q % 2 == 0:
        s += 1
        q //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    c = pow(z, q, p)
    x = pow(a, (q + 1) // 2, p)
    t = pow(a, q, p)
    m = s
    while t != 1:
        i = 1
        value = t * t % p
        while value != 1:
            value = value * value % p
            i += 1
        b = pow(c, 1 << m - i - 1, p)
        x = x * b % p
        t = t * b * b % p
        c = b * b % p
        m = i
    return x

def order(a, n):
    value = carmichael(n)
    for p, _ in groups(value):
        while value % p == 0 and pow(a, value // p, n) == 1:
            value //= p
    return value

```

```

def linear_smallest(a, b, m):
    g = math.gcd(a, m)
    if b % g:
        return None
    mod = m // g
    return b // g * pow(a // g, -1, mod) % mod if mod > 1 else 0

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(1, len(d), 3):
        a, b, m = d[i:i + 3]
        x = discrete_log(a, b, m)
        out.append(str(-1 if x is None else x))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

D. Modular Square Roots

How to find it

Use Euler classification, split $p-1=q*2^s$, and cancel the two-power discrepancy with a known nonresidue.

Correctness

Tonelli–Shanks maintains $x^2 = a*t$ and moves t into successively smaller two-power subgroups until $t=1$.

Complexity

$O(\log^2 p)$ modular operations.

C++

```

#include <bits/stdc++.h>
using namespace std;
using u64=uint64_t;
using u128=__uint128_t;
using i128=__int128_t;
u64 mul(u64 a,u64 b,u64 m) {
    return(u128)a*b%m;
}
u64 power(u64 a,u64 e,u64 m) {
    u64 r=1%m;
    for(;e>=1,a=mul(a,a,m))if(e&1)r=mul(r,a,m);
    return r;
}
bool prime(u64 n) {
    if(n<2)return false;
    for(u64 p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
    u64 d=n-1,s=0;
    while(!(d&1))d>>=1,++s;
    for(u64 a:{2,325,9375,28178,450775,9780504,1795265022}) {
        if(a%n==0)continue;
        u64 x=power(a,d,n);
        if(x==1||x==n-1)continue;
        bool witness=true;
        for(u64 r=1;r<s;r++) {

```

```

        x=mul(x,x,n);
        if(x==n-1) {
            witness=false;
            break;
        }
    }
    if(witness)return false;
}
return true;
}
mt19937_64 rng(712367821);
u64 rho(u64 n) {
    if(n%2==0)return 2;
    if(n%3==0)return 3;
    while(true) {
        u64 c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
        while(d==1) {
            x=(mul(x,x,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            y=(mul(y,y,n)+c)%n;
            d=gcd(x>y?x-y:y-x,n);
        }
        if(d!=n)return d;
    }
}
vector<u64> factor(u64 n) {
    vector<u64>out,st= {
        n
    }
    ;
    while(!st.empty()) {
        u64 x=st.back();
        st.pop_back();
        if(x==1)continue;
        if(prime(x))out.push_back(x);
        else {
            u64 d=rho(x);
            st.push_back(d);
            st.push_back(x/d);
        }
    }
    sort(out.begin(),out.end());
    return out;
}
vector<pair<u64,int>>groups(u64 n) {
    vector<pair<u64,int>>g;
    for(u64 p:factor(n))if(!g.empty()&&g.back().first==p)g.back().second++;
    else g.push_back({p,1});
    return g;
}
u64 phi(u64 n) {
    u64 r=n;
    for(auto[p,e]:groups(n))r=r/p*(p-1);
    return r;
}
u64 lcm64(u64 a,u64 b) {
    return a/gcd(a,b)*b;
}
u64 lambda(u64 n) {
    u64 r=1;
    for(auto[p,e]:groups(n)) {
        u64 z;
        if(p==2&&e>=3)z=1ULL<<(e-2);

```

```

        else {
            z=p-1;
            for(int i=1;i<e;i++)z*=p;
        }
        r=lcm64(r,z);
    }
    return r;
}
u64 primitive(u64 p) {
    if(p==2)return 1;
    vector<u64>q;
    for(auto [x,e]:groups(p-1))q.push_back(x);
    for(u64 g=2;;g++) {
        bool ok=true;
        for(u64 x:q)if(power(g,(p-1)/x,p)==1) {
            ok=false;
            break;
        }
        if(ok)return g;
    }
}
long long invmod(long long a,long long m) {
    long long b=m;
    i128 x=1,y=0;
    while(b) {
        long long q=a/b,t=a%b;
        a=b;
        b=t;
        i128 z=x-(i128)q*y;
        x=y;
        y=z;
    }
    x%=m;
    if(x<0)x+=m;
    return(long long)x;
}
optional<u64>bsgs(u64 a,u64 b,u64 m) {
    if(m==1)return 0;
    u64 z=sqrtl((long double)m)+1;
    while((u128)z*z<m)z++;
    unordered_map<u64,u64>baby;
    u64 value=1;
    for(u64 j=0;j<z;j++) {
        if(!baby.count(value))baby[value]=j;
        value=mul(value,a,m);
    }
    u64 step=invmod(power(a,z,m),m);
    value=b;
    optional<u64>best;
    for(u64 i=0;i<=z;i++) {
        auto it=baby.find(value);
        if(it!=baby.end()) {
            u64 candidate=i*z+it->second;
            if(!best||candidate<*best)best=candidate;
        }
        value=mul(value,step,m);
    }
    return best;
}
optional<u64>dlog(u64 a,u64 b,u64 m) {
    a%=m;
    b%=m;
    if(b==1%m)return 0;

```

```

u64 added=0,scale=1;
while(true) {
    u64 g=gcd(a,m);
    if(g==1)break;
    if(b%g)return {
    }
    ;
    m/=g;
    b/=g;
    scale=mul(scale,a/g,m);
    added++;
    if(scale==b)return added;
    if(m==1)return added;
}
u64 target=mul(b,invmod(scale,m),m);
auto tail=bsgs(a%m,target,m);
if(!tail)return {
}
;
return added+*tail;
}
optional<u64>tonelli(u64 a,u64 p) {
    a%=p;
    if(a==0)return 0;
    if(power(a,(p-1)/2,p)!=1)return {
    }
    ;
    if(p%4==3)return power(a,(p+1)/4,p);
    u64 q=p-1,s=0;
    while(!(q&1))q>>=1,++s;
    u64 z=2;
    while(power(z,(p-1)/2,p)!=p-1)z++;
    u64 c=power(z,q,p),x=power(a,(q+1)/2,p),t=power(a,q,p),m=s;
    while(t!=1) {
        u64 i=1,v=mul(t,t,p);
        while(v!=1)v=mul(v,v,p),i++;
        u64 h=power(c,1ULL<<(m-i-1),p);
        x=mul(x,h,p);
        t=mul(t,mul(h,h,p),p);
        c=mul(h,h,p);
        m=i;
    }
    return x;
}
u64 order(u64 a,u64 n) {
    u64 r=lambda(n);
    for(auto [p,e]:groups(r))while(r%p==0&&power(a,r/p,n)==1)r/=p;
    return r;
}
optional<u64>linear(u64 a,u64 b,u64 m) {
    u64 g=gcd(a,m);
    if(b%g)return {
    }
    ;
    u64 mod=m/g;
    if(mod==1)return 0;
    return mul((b/g)%mod,invmod((a/g)%mod,mod),mod);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin>>q;

```

```

while(q--) {
    u64 p,a;
    cin>>p>>a;
    auto x=tonelli(a,p);
    if(!x) {
        cout<<"NONE\n";
        continue;
    }
    u64 y=(p-x)%p;
    if(*x==y)cout<<*x<<'\\n';
    else cout<<min(*x,y)<<' ' <<max(*x,y)<<'\\n';
}
}

```

Python

```

import sys, math, random
MR_BASES = (2, 325, 9375, 28178, 450775, 9780504, 1795265022)

def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2
    for a in MR_BASES:
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
_rng = random.Random(712367821)

def rho(n):
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3
    while True:
        c = _rng.randrange(1, n)
        x = _rng.randrange(0, n)
        y = x
        d = 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = math.gcd(abs(x - y), n)
        if d != n:
            return d

```

```

def factor(n):
    out = []
    stack = [n]
    while stack:
        x = stack.pop()
        if x == 1:
            continue
        if is_prime(x):
            out.append(x)
        else:
            d = rho(x)
            stack.extend((d, x // d))
    out.sort()
    return out

def groups(n):
    result = []
    for p in factor(n):
        if result and result[-1][0] == p:
            result[-1] = (p, result[-1][1] + 1)
        else:
            result.append((p, 1))
    return result

def phi(n):
    answer = n
    for p, _ in groups(n):
        answer = answer // p * (p - 1)
    return answer

def lcm(a, b):
    return a // math.gcd(a, b) * b

def carmichael(n):
    answer = 1
    for p, e in groups(n):
        part = 1 << e - 2 if p == 2 and e >= 3 else (p - 1) * p ** (e - 1)
        answer = lcm(answer, part)
    return answer

def primitive_root(p):
    if p == 2:
        return 1
    primes = [q for q, _ in groups(p - 1)]
    g = 2
    while any((pow(g, (p - 1) // q, p) == 1 for q in primes)):
        g += 1
    return g

def bsgs(a, b, m):
    a %= m
    b %= m
    if m == 1:
        return 0
    size = math.isqrt(m) + 1
    baby = {}
    value = 1
    for j in range(size):
        baby.setdefault(value, j)
        value = value * a % m
    step = pow(pow(a, size, m), -1, m)
    value = b
    best = None

```

```

for i in range(size + 1):
    if value in baby:
        candidate = i * size + baby[value]
        if best is None or candidate < best:
            best = candidate
    value = value * step % m
return best

def discrete_log(a, b, m):
    a %= m
    b %= m
    if b == 1 % m:
        return 0
    added = 0
    scale = 1
    while True:
        g = math.gcd(a, m)
        if g == 1:
            break
        if b % g:
            return None
        m //= g
        b //= g
        scale = scale * (a // g) % m
        added += 1
        if scale == b:
            return added
        if m == 1:
            return added
    target = b * pow(scale, -1, m) % m
    tail = bsgs(a % m, target, m)
    return None if tail is None else added + tail

def tonelli(a, p):
    a %= p
    if a == 0:
        return 0
    if pow(a, (p - 1) // 2, p) != 1:
        return None
    if p % 4 == 3:
        return pow(a, (p + 1) // 4, p)
    q = p - 1
    s = 0
    while q % 2 == 0:
        s += 1
        q //= 2
    z = 2
    while pow(z, (p - 1) // 2, p) != p - 1:
        z += 1
    c = pow(z, q, p)
    x = pow(a, (q + 1) // 2, p)
    t = pow(a, q, p)
    m = s
    while t != 1:
        i = 1
        value = t * t % p
        while value != 1:
            value = value * value % p
            i += 1
        b = pow(c, 1 << m - i - 1, p)
        x = x * b % p
        t = t * b * b % p
        c = b * b % p

```

```

        m = i
    return x

def order(a, n):
    value = carmichael(n)
    for p, _ in groups(value):
        while value % p == 0 and pow(a, value // p, n) == 1:
            value //= p
    return value

def linear_smallest(a, b, m):
    g = math.gcd(a, m)
    if b % g:
        return None
    mod = m // g
    return b // g * pow(a // g, -1, mod) % mod if mod > 1 else 0

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    out = []
    for i in range(1, len(d), 2):
        p, a = d[i:i + 2]
        x = tonelli(a, p)
        if x is None:
            out.append('NONE')
        else:
            roots = sorted({x, (p - x) % p})
            out.append(' '.join(map(str, roots)))
    print('\n'.join(out))
if __name__ == '__main__':
    main()

```

Proof and stress companion

Primitive-root tests use only distinct prime divisors of $p-1$: a candidate has full order exactly when none of those divisions leaves exponent one. BSGS must return the smallest legal exponent, not merely a collision. In the non-coprime version, test early solutions while gcd reductions are performed; only invoke inverse-based BSGS once the reduced base is a unit.

For Tonelli–Shanks, first classify zero and non-residues. Verify every returned root by squaring and normalize the requested representative. Exhaustively enumerate exponents and residues for small moduli, including $x=0$, $b=1$, nonunits, repeated gcd reductions, and no-solution instances.